

The Four Irreducibilities of Neural Proof Search

Structural Constants, Impossibility Theorems, and Architectural Prescriptions
for Obligation-Driven Completeness in AI Mathematical Reasoning

Vario

Vario Automation — Mnehmos

Graham County, Arizona

February 2026 — Consolidated Research Program v2

DARPA-PA-25-07-02 (CLARA) | Track A/B Research Program | The Consent Horizon Project

Abstract

This document establishes the theoretical, empirical, and architectural foundations for a research program demonstrating that neural proof search systems are bounded by four irreducible structural constraints: **Obligation Opacity**, **Solver Opacity**, the **Verification-Exploration Tradeoff**, and **Strategy Interference**. These constraints are not engineering bottlenecks awaiting better hardware or architectures. They are mathematical barriers grounded in proven impossibility theorems from computability theory, computational complexity, information theory, and optimization.

The architectural response to these constraints is the Obligation Engine—an AR-primary orchestration layer whose central design innovation is the separation of storage from attention. A directed acyclic graph (DAG) persists all proof state, while each agent call receives only a surgically curated context slice. Everything persists; only what matters activates. Agents operate as stateless functions over the database. The orchestration layer decides what each agent sees. The database is the intelligence; the agents are hands.

The engineering argument parallels the theoretical one. Empirical measurement shows that approximately 77% of tokens in LLM reasoning chains are causally decoupled from terminal decisions (Project Ariadne, 2025). Current architectures waste compute on decorative reasoning at inference time, contaminated gradient signals at training time, and redundant tool calls at infrastructure time. This document presents a unified waste-elimination framework across three substrates—training (Track B: shaping the loss landscape), inference (the Obligation Engine: curating context per call), and infrastructure (Scalpel patterns: eliminating retries and scope at the tool level)—applying the same hierarchy of controls at each layer.

Each constraint is stated as a falsifiable claim with explicit experimental protocols, formal impossibility arguments, architectural prescriptions, and conditions under which we would declare it falsified. All evidence converges from three independent analytical streams: formal impossibility results, empirical measurements from frontier AI systems, and cross-validation by independent AI systems analyzing the same claim set.

1. The Case for Impossibility Results

Shannon’s channel capacity theorem did not discourage communications engineering. It redirected it. Engineers stopped building error-free channels and started building error-correcting codes. Impossibility results do not prevent progress. They *focus* it by distinguishing what engineering can solve from what architecture must accommodate.

The IMO 2025 competition provides a clean demonstration. Google DeepMind’s Gemini with Deep Think, OpenAI’s reasoning models, and ByteDance’s Seed-Prover all achieved gold-medal aggregate scores by solving five of six problems. Every system failed Problem 6, a combinatorial optimization requiring exhaustive coverage of a permutation space. The diagnostic: each system verified individual reasoning steps with high accuracy using Lean 4 or CAS, but lacked the mechanism to verify complete case coverage. Systems converged on the incorrect answer 4048 (correct: 2112) because they treated unexplored permutation cases as handled by default—collapsing the unknown into the known.

DARPA’s CLARA program identifies this precisely: the industry standard of “tacking on” automated reasoning to large language models produces weak assurance. CLARA calls for a fundamental inversion—AR-primary architectures where automated reasoning orchestrates and machine learning generates within structural constraints. The four irreducibilities explain *why* this inversion is necessary, and the waste-elimination framework explains *how* to implement it economically.

2. The 77% Problem: Quantifying Waste in Neural Reasoning

Before presenting the four irreducibilities, we establish the empirical baseline that motivates the entire architectural program.

Project Ariadne (2025, arXiv:2601.02314) applied do-calculus interventions to LLM chain-of-thought reasoning traces and measured a **violation density of 0.767**: approximately 77% of reasoning steps are causally decoupled from the terminal decision. The model appears to reason step by step, but surgically altering most intermediate steps does not change the final answer. The steps look like reasoning, are structured like reasoning, and pass surface-level inspection—but they are not load-bearing. The model committed to a trajectory early and generated plausible intermediate work to fill the space between commitment and output.

This result has cascading implications across the full development stack:

At inference time: 77% of tokens in a reasoning chain are waste. Every unnecessary tool call multiplies this—the call’s tokens *plus* the decorative reasoning the model generates about what it retrieved. A 1000-token chain contains roughly 230 load-bearing tokens and 770 tokens of articulate filler. Current architectures cannot distinguish which is which.

At training time: Every RLHF run, process reward model, and “let’s verify step by step” pipeline trains on both the 23% that computes and the 77% that decorates. Gradient signals from decorative tokens pull the loss landscape in directions unrelated to reasoning. The landscape itself is shaped by noise.

At infrastructure time: Redundant API calls, unbounded context windows, and retry loops from ambiguous inputs each trigger downstream chains of decorative reasoning. One unnecessary file

read costs not just the file tokens but the 770 decorative tokens per 1000 that the model generates about what it read.

The Obligation Engine, Track B training methods, and Scalpel infrastructure patterns are three responses to the same quantified problem: most of what current systems compute is waste, and the waste compounds across layers.

3. The Four Irreducibilities: Overview

Each irreducibility is stated as a falsifiable claim grounded in established impossibility theorems, supported by empirical measurement, and mapped to a governance analog from the Consent Horizon framework.

Irreducibility	Formal Claim	Primary Theorem	Governance Analog
Obligation Opacity	The complete proof obligation set cannot be determined prior to partial proof construction, and this opacity recurs at every node in the proof tree	Cut-elimination blowup (Statman 1979; Boolos 1984); Rice’s theorem for abstract semantics (Baldan et al. 2021); exponential separations for neural provers (arXiv:2602.10512)	Lossy Intent: no representation of volition captures full dimensionality
Solver Opacity	Verification of outputs does not yield a predictive model of the generative process; 77% of intermediate steps are causally decorative	NP-hardness of circuit discovery (Adolfi et al. 2024); interpretability queries Σ_2^P -complete (Barceló et al. 2020); Complexity Gap Theorem (arXiv:2504.20676)	Incomplete Self-Knowledge: no agent fully models its own process
V-E Tradeoff	Maximizing verification depth strictly reduces exploration breadth; Pareto frontier is strictly concave and provably tight	Tight Pareto frontier $R_t \cdot S_t = \Theta(T^3)$ (Chen et al. 2026); Lai-Robbins lower bound (1985)	Temporal Validation Cost: the mechanism that builds trust consumes the resource it protects
Strategy Interference	Pursuit of one proof path forecloses alternatives by strictly positive switching cost; context contamination is universal and architectural	Minimax regret $\Theta(K \cdot T^2)$ with switching costs (Dekel et al. 2014); universal context rot (Chroma 2025)	Concurrent Interests: legitimate alternatives compete for finite attention

4. Irreducibility I: Obligation Opacity

The proof doesn’t know what it needs until it’s partially built—at every depth.

4.1 The Claim

For any non-trivial mathematical statement S , the complete set of proof obligations $O(S) = \{o_1, o_2, \dots, o_n\}$ cannot be enumerated by any computable function operating solely on S . This opacity is not limited to the root: it recurs at every node in the proof tree. Standing at step k with obligation o_i , the sub-obligations required to close o_i are themselves opaque until the proof below o_i is partially constructed. The problem statement is a lossy compression of its proof space, and each obligation is a lossy compression of its own subtree.

This is the **recursive case** of obligation opacity. Take any vertical slice of the proof tree: there are an arbitrary number of nodes where the required sub-obligations cannot be known until the verification at that node collapses. Before collapse, the subtree is an open space of possibilities. After verification, it crystallizes into a single closed fact. The transition is instantaneous and irreversible—and it may reveal new obligations at adjacent nodes that were literally unknowable before the collapse occurred.

4.2 Theoretical Foundations

Cut-elimination blowup (Theorem). Statman (1979) and Orevkov (1979) proved that eliminating lemmas (cuts) from proofs produces a superexponential blowup—a tower of exponentials whose height scales with cut depth. Buss (2012) sharpened this: proofs of height $O(d)$ with cuts yield cut-free proofs requiring size $\geq 2^{2^{\dots}}$ (tower of height $d - O(1)$). Boolos (1984) constructed derivations completable in a page using cuts whose analytic proofs could not be completed in the lifespan of the universe. The cuts—which correspond to lemmas introducing concepts outside the vocabulary of the theorem statement—are computationally essential structures that emerge during proof construction.

Rice’s theorem for abstract semantics (Theorem). Baldan, Ranzato, and Zhang (ICALP 2021) extended Rice’s theorem to abstract interpretations: any nontrivial abstract property is undecidable and any decidable overapproximation includes infinitely many false positives. If obligation extraction is modeled as deciding a semantic property of the theorem-as-specification, even approximate enumeration must be either incomplete or contain infinitely many spurious obligations.

Exponential separations for neural provers (Theorem). “Don’t Eliminate Cut” (arXiv:2602.10512, February 2025) models LLM-guided proving as a stochastic policy in a finite-horizon MDP and proves exponential separations between provers that can introduce lemmas and those restricted to the theorem’s vocabulary. The obligation space includes lemmas that cannot be extracted from the statement—formalizing obligation opacity for neural systems.

Architectural evidence. Every major proof assistant generates obligations incrementally via tactic application. Coq’s `apply` tactic returns as many subgoals as the number of non-dependent premises; these become known only after application. Every successful AI prover (AlphaProof, Seed-Prover, DeepSeek-Prover) uses tree search precisely because proof structure cannot be predicted from the statement. Kohlenbach’s proof mining extracts computational content that was not visible beforehand—and different proofs of the same theorem yield different extracted bounds.

4.3 Empirical Measurement

ChatDB prototype measurements on IMO 2020–2025 problems yield an obligation discovery ratio of 0.3–0.5 at the root level: 30–50% of obligations are identifiable only after proof exploration has

commenced. For problems requiring induction or case analysis beyond four cases, the ratio exceeds 0.5. The recursive case has not yet been systematically measured—measuring discovery ratios at internal nodes is a Phase I priority.

4.4 Experimental Protocol

Experiment 1A (Root-Level Discovery Rate): Run the obligation engine on 50 benchmark problems (IMO 2020–2025, Putnam 2020–2025, selected number theory and combinatorics). Record O_0 (obligations from statement alone), O_k (obligations at step k), O_{final} . Compute discovery ratio $|O_{\text{final}} - O_0| / |O_{\text{final}}|$. Include trivial problems as controls.

Experiment 1B (Recursive Discovery Rate): For each node in the obligation DAG, measure the local discovery ratio: what fraction of sub-obligations were identifiable before beginning work on the parent obligation versus after partial construction. This tests the recursive claim directly.

Experiment 1C (Expert Baseline): Present 50 statements to three graduate-level mathematicians. Ask each to enumerate all proof obligations before attempting the proof. Compare human pre-enumeration coverage to O_{final} . If experts achieve perfect pre-enumeration, the irreducibility is model-specific.

Experiment 1D (Scaling Analysis): Run obligation extraction with frontier models of increasing capability. Plot discovery ratio against model capability. Expected: ratio decreases with scale but does not converge to zero.

Falsification condition: If the obligation discovery ratio (root and recursive) converges monotonically to zero with model scale, achieving < 0.05 across all problem classes, then obligation opacity is a capability limitation, not structural.

4.5 Architectural Prescription

If obligation opacity is structural and recursive: obligation sets must be mutable at every node in the proof DAG, not just at the root. The DAG’s event-sourced architecture handles this natively—new obligations can be inserted at any node when a verification collapse reveals them. The completeness invariant requires closure of all *identified* obligations, not all obligations. The gap between “all obligations closed” and “all identified obligations closed” is the operational encoding of obligation opacity at every depth.

5. Irreducibility II: Solver Opacity

Verification of outputs does not imply understanding of process.

5.1 The Claim

For any neural solver S with parameters θ , external verification of S 's outputs on input set I does not yield a model M of S 's generative process sufficient to predict S 's behavior on inputs outside I with bounded error. The mapping from θ to output-generating-process is information-theoretically lossy at every feasible resolution. Empirically, approximately 77% of the solver's intermediate reasoning tokens are causally decoupled from its decisions—the solver's own chain of thought is an unreliable model of its own process.

5.2 Theoretical Foundations

NP-hardness of circuit discovery (Theorem). Adolphi et al. (2024, arXiv:2410.08025) proved that circuit discovery—the core method of mechanistic interpretability—is NP-hard. Even with full model access, identifying the computational circuits responsible for specific behaviors is intractable assuming $P \neq NP$.

Interpretability queries are Σ_2^P -complete (Theorem). Barceló, Monet, Pérez, and Subercaseaux (NeurIPS 2020) proved that minimum-change-required is NP-complete for MLPs, counting-completions is $\#P$ -complete, and minimum-sufficient-reason is Σ_2^P -complete.

Complexity Gap Theorem (Theorem). arXiv:2504.20676 (2025) uses Kolmogorov complexity to prove that any explanation significantly simpler than the original model must necessarily differ from it on some inputs. Rudin (2019, Nature Machine Intelligence): if an explanation were completely faithful to the model, the explanation would equal the model.

No Free Lunch (Theorem). Wolpert and Macready (1997) proved that any two algorithms are equivalent when performance is averaged across all possible problems. Without structural assumptions, observed outputs give zero information about novel-input behavior.

Representational interference (Empirical). Bochkov et al. (TMLR 2025, arXiv:2507.04886) demonstrated that frozen visual Unicode embeddings outperform trainable embeddings by nearly $2\times$ on MMLU (23.81 vs. 12.54). Training introduces representational interference—the optimization process degrades the structure it exploits. The same inputs produce coherent outputs through radically different internal representations, making the generative process non-invertible from outputs alone.

Causal decoupling (Empirical). Project Ariadne (2025) measured a violation density of 0.767—roughly 77% of reasoning steps are causally decoupled from terminal decisions. The gap between explainable-after-the-fact and predictable-before-the-fact is the operational definition of solver opacity.

Post-hoc explanation failure (Empirical). Slack et al. (2020) showed LIME and SHAP can be fooled in 84–100% of cases. Anthropic's circuit tracing work (2025) reveals the current frontier of mechanistic interpretability: features are imperfectly described, manual analysis is required, and “dark matter” features remain beyond current methods.

5.3 Implications for the 77% Problem

The 77% causal decoupling result has a specific architectural consequence: the solver’s own chain of thought cannot be used as evidence for what it “knew” or “considered.” If an obligation engine relies on solver self-annotation to identify obligations (Layer 1 in the hybrid parsing pipeline), it captures approximately 10% of semantic obligations—and the self-annotations themselves may be causally decorative. This is why the architecture requires CAS-generated obligations (Layer 4, ~95% recall) as the ground truth and treats solver self-reports as supplementary at best.

For training, the implication is equally direct: models trained on chain-of-thought data are training on 77% noise. Every gradient update from a decorative token pulls the loss landscape in a direction unrelated to reasoning. Track B’s typed tokenization, curated pedagogy, and strategic parameter freezing are responses to this quantified problem—they shape the gradient landscape so updates reflect the 23% that computes, not the 77% that decorates.

5.4 Experimental Protocol

Experiment 2A (Output Verification vs. Process Prediction): For each verified-correct step in ChatDB’s data, attempt to predict whether the same solver will produce a correct step on a structurally similar novel sub-problem. Build a prediction model from output history. Expected: prediction accuracy plateaus below 0.85.

Experiment 2B (Cross-Model Opacity): Run identical problems through multiple foundation models with CAS/Lean verification. Test whether knowing Model A’s error pattern predicts Model B’s. Expected: low cross-model error correlation for most problem types.

Experiment 2C (Causal Load-Bearing Analysis): Apply Ariadne-style do-calculus interventions to ChatDB solver traces. For each step, measure whether surgically altering it changes the solver’s subsequent outputs. Map the load-bearing fraction to obligation closure events. Expected: load-bearing steps cluster around obligation state changes, with decorative steps filling the intervals.

Falsification condition: If an interpretability method achieves >0.9 causal prediction accuracy on novel proof steps across multiple domains, solver opacity is a current limitation, not structural.

5.5 Architectural Prescription

If solver opacity is structural: **never trust the solver’s process; verify only its outputs.** The separation of solver (ML) from verifier (AR) is not a defense against bad models—it is an acknowledgment that no model, regardless of quality, can be trusted on the basis of its process. The six-category training signal taxonomy exists because some outputs resist verification and the architecture must be honest about which it has verified and which it has merely observed.

6. Irreducibility III: The Verification-Exploration Tradeoff

The mechanism that builds trust also consumes the resource it protects.

6.1 The Claim

For any fixed computational budget B , maximizing verification depth V strictly reduces exploration breadth E . The Pareto frontier between V and E is strictly concave and provably tight: no allocation achieves maximum V and maximum E simultaneously.

6.2 Theoretical Foundations

Tight Pareto frontier (Theorem). Chen et al. (February 2026, arXiv:2602.07472) established a tight product constraint between regret and allocation variability in multi-armed bandits: $R_t \cdot S_t = \Theta(T^{3/2})$. Simultaneously stable and minimax regret-optimal algorithms provably do not exist. The frontier is a hyperbola—strictly concave.

Lai-Robbins lower bound (Theorem). Any consistent bandit algorithm must suffer regret at least $\Omega(\log n)$, with each suboptimal arm requiring $\Omega(\log n / D_{\text{KL}})$ pulls. Auer, Cesa-Bianchi, and Fischer (2002) showed UCB1 matches this up to constants, proving the bound is tight. Exploration necessarily costs exploitation.

Verification cost scaling. Type checking in dependent type theory depends on normalization; Wells (1999) proved typability in System F is undecidable. Erbsen et al. (2024) showed proof rewriting in Coq can produce exponential blowup. Hoeffding’s inequality: statistical verification accuracy scales as $1/\sqrt{n}$ —quadratic cost to halve the error.

Production systems evidence. AlphaProof required multi-day computation on thousands of TPUs. Seed-Prover capped maximum search depth at 4 while increasing parallel width. Snell et al. (2024) measured diminishing returns: improvements greatly diminish as test-time search budget scales.

6.3 The Muda Connection

The V-E tradeoff explains why waste elimination is the competitive strategy against brute-force scaling. If the Pareto frontier is strictly concave, then every wasted token moves the system further from the frontier than the same token’s contribution would move it along the frontier. Eliminating the 77% of decorative reasoning tokens doesn’t just save compute—it moves the system’s operating point closer to the Pareto frontier by concentrating budget on load-bearing computation.

Scale compute solves the tradeoff by expanding the budget. Muda elimination solves it by moving the operating point toward the frontier within a fixed budget. Both work. On a \$40–60 per-problem API budget, only the second is available.

6.4 Experimental Protocol

Experiment 3A (Budget Allocation Sweep): Run the obligation engine on 30 problems with allocation parameter $\alpha \in [0, 1]$ where $\alpha = 1$ means all budget on verification and $\alpha = 0$ means all

budget on exploration. Record correctness rate, completeness rate, and total resource consumption. Expected: U-shaped failure curve with optimal zone around $\alpha \in [0.3, 0.6]$.

Experiment 3B (Verification Cost Scaling): For a single problem, measure token cost at each verification level: syntactic, CAS, Lean formalization, Lean with supporting lemmas. Expected: super-linear cost scaling.

Experiment 3C (Waste-Eliminated vs. Standard): Compare standard long-chain reasoning against ChatDB's stateless short calls on identical problems and identical total token budgets. Measure: correctness per token, obligations closed per token, Pareto frontier position. Expected: waste-eliminated architecture operates closer to the frontier.

Statistical method: Nonlinear mixed-effects models (problem-level random intercepts) for correctness(B) and obligation mass $M(B)$. Survival analysis for obligation closure time distributions. Compare saturating, log-linear, and power-law fits via information criteria.

Falsification condition: If any allocation achieves both maximum verification confidence and maximum obligation coverage within a fixed budget, the tradeoff is an allocation problem, not structural.

6.5 Architectural Prescription

If the tradeoff is structural: **make the allocation visible and auditable.** The obligation engine's audit trail records how budget was allocated across branches. The approximation chain from radial restraint formalizes iterative improvement: each pass reallocates budget based on what the previous pass revealed. The system does not pretend to verify everything. It makes its verification coverage—its approach velocity toward completeness—explicit.

7. Irreducibility IV: Strategy Interference

Pursuing one proof path forecloses others.

7.1 The Claim

For a proof search problem with K distinct valid strategies, pursuit of strategy s_i reduces the expected utility of switching to strategy s_j by a strictly positive cost $c(i,j) > 0$ for $i \neq j$. This cost is not a property of the solver’s capability but of the context’s contamination: accumulated working state from strategy s_i primes the solver toward continuation rather than divergence. No scheduling algorithm simultaneously maximizes coverage breadth, verification depth, and time efficiency for arbitrary proof trees.

7.2 Theoretical Foundations

Switching cost regret gap (Theorem). Dekel, Ding, Koren, and Peres (ICML 2014) proved the minimax regret of K -armed bandits with unit switching costs is $\Theta(K^{1/3} \cdot T^{2/3})$, versus $\Theta(\sqrt{T})$ without switching costs. The gap from $T^{1/2}$ to $T^{2/3}$ is proven and irreducible—any algorithm guaranteeing $O(\sqrt{T})$ regret must make $\Omega(T)$ switches. Koren, Livni, and Mansour (2017) generalized to metric movement costs. Amir et al. (NeurIPS 2022) confirmed the interference is fundamental even in stochastic settings.

Context contamination (Empirical). Liu et al. (2024, TACL) demonstrated the “Lost in the Middle” effect: LLM performance degrades by over 30% when relevant information appears mid-context. Chroma Research (2025) tested 18 frontier models and found universal context rot at every context length increment. The mechanism is architectural: rotary position embedding introduces long-term decay that de-emphasizes mid-context tokens.

Commitment irreversibility. Alpha-beta pruning (Knuth and Moore 1975): with optimal move ordering, search examines $O(b^{d/2})$ nodes; with pessimal ordering, $O(b^d)$ —exhaustive search. Yokoo (1994) formalized backtracking costs: bad partial solutions require exhaustive search for revision.

Multi-objective impossibility. Eck and Pinedo (1993, Operations Research) proved makespan and total flowtime are mathematically conflicting objectives. Arrow’s impossibility theorem provides the structural analog: with three or more alternatives, no aggregation satisfies unanimity, independence of irrelevant alternatives, and non-dictatorship.

7.3 The Conversational Instance

Strategy interference is directly observable in ordinary human-AI conversation. Users make statements; models compound on them; context accumulates along a single trajectory. The longer the conversation runs, the more the model’s outputs converge toward the user’s established frame—not because the model agrees, but because the accumulated context primes continuation over divergence. This is context contamination masquerading as alignment.

The non-harmful case is productive: building a coherent document benefits from compounding momentum. The harmful case is when refutation is needed and the context makes it structurally unlikely. The architectural response—the Adversary agent, anti-context retrieval, fresh-context

branching—addresses both the proof-search case and the conversational case through the same mechanism: cutting a new context that doesn’t carry the prior trajectory’s contamination.

7.4 Experimental Protocol

Experiment 4A (Strategy Switching Cost): Compare Strategy B success rate when context contains Strategy A’s working state versus with clean context. The difference is the empirical switching cost. Expected: significant degradation proportional to strategy divergence.

Experiment 4B (Parallel vs. Sequential): Compare sequential, round-robin, and independent parallel (separate context windows). Expected: independent parallel outperforms both but consumes $K\times$ budget. Eliminating interference requires multiplying the budget.

Experiment 4C (Conversational Instance): Run the same proof problem as a single long conversation versus as a sequence of fresh-context calls with only verified state carried forward. Measure correctness, diversity of approaches attempted, and ratio of load-bearing to decorative tokens.

Statistical method: Pairwise Jaccard overlaps of explored technique families across (solver $\times$ paraphrase $\times$ run). Permutation tests for overlap significance. ECE-style calibration metrics where models output confidence.

Falsification condition: If a strategy scheduler achieves maximum coverage, depth, and efficiency simultaneously on arbitrary proof trees, strategy interference is an algorithm design problem, not structural.

7.5 Architectural Prescription

If strategy interference is structural: **make strategy choices explicit and reversible where possible.** The demotion protocol with mandatory working notes documents every failed approach. The DAG’s event-sourced architecture ensures no work is lost. The Librarian agent’s anti-context retrieval addresses contamination structurally. And the critical insight: the default operating mode is stateless short calls with curated minimal context. Accumulation mode—allowing the solver to build momentum within a productive branch—is the exception triggered by sustained progress, not the default.

8. The Central Design Innovation: Separation of Storage from Attention

The four irreducibilities converge on a single architectural requirement: **the linear context window is the wrong data structure for proof search.** Append-only context forces chronological accumulation—every step taken sits in front of the solver whether relevant or not. This is the structural mechanism through which strategy interference operates, the channel through which decorative tokens accumulate, and the reason verification and exploration compete rather than compose.

The Obligation Engine’s directed acyclic graph inverts this relationship. **Everything persists in the database. Only what matters enters the context.** The DAG is the single source of truth: all obligations, all verified results, all working notes, all failed approaches. But no agent ever sees the full DAG. Each agent call receives a surgically curated context slice constructed by the orchestration layer for that specific call.

8.1 Two Opposite Context Policies

The architecture does not have a single context policy. It has a spectrum, with two principal operating points:

Minimal context (default for new branches and verification): The agent receives the obligation definition, the relevant formal definitions, and the constraint set. No history of what other agents tried, no accumulated working notes, no failed approaches. One clean shot. This is the correct policy when the information needed for the next step doesn’t yet exist (obligation opacity) or when prior context would introduce bias (strategy interference, verification independence).

Accumulation context (triggered by sustained progress): When a solver is actively closing obligations—when the plateau detector shows positive obligation-closure velocity—the orchestration layer allows context to accumulate within that branch. The solver builds on its own intermediate results. This is productive compounding: the generative capacity working as intended. The trigger to cut back to minimal context is the plateau—when steps are being produced but obligations aren’t closing.

The transition between modes is the intelligence of the system. Not “you’ve been running too long” but “you’ve stopped making progress.” Some problems need long chains. Some need fresh eyes after three steps. The plateau detector, not a timer, makes this decision.

8.2 Agents as Stateless Functions

Every agent call—solver, verifier, adversary, librarian, formalizer—is a stateless function over a curated slice of the database. The agent receives a context packet, produces an output, and terminates. The orchestration layer (the OODA loop) observes the output, updates the DAG, orients against the obligation graph, decides the next assignment, and constructs the next context packet.

The OODA loop runs at the orchestration layer. The agents never orient. They never observe their own history. They are pure functions. This is not just an efficiency choice—it is the structural

response to solver opacity. If 77% of an agent’s reasoning is causally decorative, the only honest architecture treats each call as independent and verifies its output rather than trusting its chain.

8.3 Why This Explains the Cost Model

The per-problem cost of \$40–60 is not paying for one long reasoning chain with an enormous context window. It is paying for many short, cheap, stateless calls with surgically minimal context. Each call is fast, focused, and independently verifiable. The architecture trades one expensive deep chain (dominated by 77% decorative tokens) for many cheap shallow probes (each targeting a specific obligation). The DAG is what gives them coherence without requiring them to share context.

This is muda elimination at the architecture level: the seven wastes of linear-context reasoning are structurally impossible in the DAG architecture.

Waste (Muda)	In Linear Context	In DAG Architecture
Overproduction	Solver generates steps past plateau	Plateau detector cuts to new branch
Waiting	Solver blocked during irrelevant verification	Stateless parallel calls; no blocking
Transport	Context carries irrelevant history	Curated slices; only relevant state enters context
Overprocessing	All steps verified at maximum depth	Three-level verification matches effort to need
Inventory	Accumulated context never used again	Context trimmed at branch points; DAG stores everything
Motion	Strategy switching through contaminated context	Fresh context for each new branch
Defects	Verified-but-incomplete proofs submitted	Completeness invariant gates termination

9. The Three-Substrate Waste-Elimination Stack

The 77% problem manifests at three layers, and each layer requires its own intervention. The hierarchy of controls—elimination beats substitution beats engineering controls beats administrative controls beats PPE—applies at each layer. The interventions are ordered by effectiveness, not by ease of implementation.

9.1 Training: Track B — Shaping the Loss Landscape

If 77% of chain-of-thought tokens are decorative, then models trained on chain-of-thought are training on 77% noise. Gradient signals from decorative tokens pull the loss landscape in directions unrelated to reasoning. Track B eliminates this waste at the source:

Typed tokenization (Elimination): Tokens carry type information, giving the embedding space structure before training begins. Gradient descent starts on a landscape with semantically meaningful directions already carved in. The optimizer doesn't spend billions of updates discovering structure that could be provided.

Strategic parameter freezing (Elimination): Bochkov et al. demonstrated that frozen embeddings outperform trainable ones by $2\times$ because training introduces representational interference—gradients for low-level representation and high-level reasoning pull the same parameters in opposite directions. Freezing eliminates the conflict. The geometry becomes followable because contradictory forces are removed.

Polysemy resolution (Substitution): When one region of weight space encodes multiple unrelated functions, the conflicting gradients are the parameter-level equivalent of context contamination. Restructuring tokenization so distinct semantic roles get distinct representational territory eliminates interfering gradients at the source.

Curated pedagogy (Engineering Control): Curriculum design that sequences training so each phase carves geometry the next phase can follow. Easy gradients first, building structure; complex gradients later, refining it. Train on verified state transitions, not on the decorative tokens between them.

The connection to B2 (Constraint as Information) is direct: Shannon says constraints reduce uncertainty. A constrained loss landscape has fewer directions to explore. Fewer directions means less wasted optimizer movement. Every constraint added to the geometry is information that eliminates muda from training.

9.2 Inference: The Obligation Engine — Curating Context per Call

The obligation engine eliminates waste at inference time by controlling what each agent sees. Stateless short calls with minimal context replace long chains dominated by decorative reasoning. The DAG persists state; agents don't carry it. Verification is independent of generation. Budget allocation is explicit and auditable.

The key metrics: obligations closed per token (efficiency), load-bearing token fraction (quality), and approach velocity toward completeness (progress). All three are measurable. All three are optimized by waste elimination rather than by scale.

9.3 Infrastructure: Scalpel Patterns — Eliminating Retries and Scope

The Scalpel patterns (Mnehmos, 2025) implement waste elimination at the tool layer. The token equation— $\text{Tokens} = \text{Scope} \times \text{Iterations} \times \text{Verbosity}$ —identifies three multiplicative factors. Most approaches attack only verbosity (the lowest-impact factor). The Scalpel patterns attack scope and iterations:

Fuzzy enum matching (Elimination): Three-tier resolution (exact, alias, Levenshtein) auto-corrects LLM typos before they become retry loops. 285+ synonyms mapped to canonical values across 15+ categories. ~95% retry reduction. The condition that causes retries is removed entirely.

Guiding errors (Substitution): When matching fails, the error contains the fix: “87% similar. Use this exact text.” The LLM copies exact text, succeeds on retry 1. Unbounded retry loops become bounded one-shot corrections.

Batch operations (Elimination): Every file operation has a batch variant. 20 calls → 1 call. The tokens saved are not just the 19 eliminated calls but the 19×770 decorative tokens the model would have generated reasoning about each result.

Workspace boundaries (Elimination): Explicit `in_scope/out_of_scope` constraints prevent agents from exploring outside their domain. The agent doesn’t exercise discipline—undisciplined behavior is structurally impossible. Same principle as the verifier’s minimal context: scope control by construction, not by instruction.

9.4 The Unified Frame

All three layers apply the same hierarchy of controls from the same source domain (industrial safety, where the framework was developed for situations where failure is unacceptable). The ordering is consistent: elimination > substitution > engineering controls > administrative controls > PPE. Prompt engineering (verbosity reduction) is PPE—the last resort, lowest effectiveness. Architectural constraint (DAG, typed tokens, workspace boundaries) is elimination—the first resort, highest effectiveness.

Hierarchy Level	Training (Track B)	Inference (Obligation Engine)	Infrastructure (Scalpel)
Elimination	Freeze parameters; typed tokenization	Stateless calls; DAG-curved context	Fuzzy matching; batch ops; workspace bounds
Substitution	Polysemy resolution; restructured tokens	Fresh-context branching at plateau	Guiding errors (one-shot correction)
Engineering Control	Curated curriculum sequencing	Three-level verification pipeline	Flexible identifiers (name or UUID)
Administrative Control	Training data quality filters	Obligation priority ordering	Logging and audit trails
PPE	Prompt formatting for training	Prompt engineering for agents	Response verbosity settings

10. Observed Failure Modes Mapped to Irreducibilities

Failure Mode	Where Observed	Primary Irreducibility	Mechanism
Correct local steps, wrong global answer	IMO 2025 P3; process supervision broadly	Obligation Opacity	Step verifiers validate a path; the path omits a necessary case family
Universal miss on hardest instance	IMO 2025 P6: all systems solve 5/6, fail P6	V-E Tradeoff + Strategy Interference	Demands global coverage under budget; single-trajectory reasoning is brittle

Autoformalization mismatch	miniF2F pipeline: proof for wrong statement	Obligation Opacity + Solver Opacity	Specification is lossy; LLM judges are biased
Diminishing returns / long-tail solve times	Formal provers; Snell et al. 2024	V-E Tradeoff	Heavy variance; completeness is budget-dependent
Self-correction fails	Huang et al. 2024; evaluator bias studies	Solver Opacity	Neither generators nor judges expose blind spots
77% decorative tokens	Project Ariadne 2025	Solver Opacity (cascading to all)	Most reasoning tokens are post-hoc reconstruction, not computation
Premature convergence on wrong answer	IMO P6: 4048 vs. correct 2112	Strategy Interference	Context contamination and echo chamber effects

11. Confounders and Alternative Hypotheses

11.1 Benchmark Contamination

Many popular contest sources appear in pretraining corpora. Our protocol uses IMO 2025 problems (released after most training cutoffs), PutnamBench, FormalMATH novel formalizations, and INDIMATHBENCH. Phase III paraphrase perturbation provides a direct control: if performance drops on paraphrased versions of contaminated problems but holds on clean ones, the confound is detected.

11.2 Verifier Strength Mismatch

CAS validation can be fast yet incomplete; formal proof assistants provide stronger guarantees but introduce formalization loss and library dependence. The three-level verification pipeline (syntactic, CAS, Lean) separates verification strength from structural tradeoffs.

11.3 Obligation Parser Failures

If obligation extraction is weak, “open obligations” reflect tooling gaps, not inherent barriers. This is the most dangerous confound. Phase III formalization-equivalence audits separate representation loss from search hardness. The three-layer parsing pipeline (self-tagging ~10%, ensemble 60–80%, CAS-generated 95%+) provides controlled comparison.

11.4 Current-Model Artifacts

The strongest objection: irreducibilities are artifacts of current models. Counterpoint: undecidability and intractability results bound universal guarantees. The hypothesis can hold only for restricted domains. Experiment 1D (scaling analysis) directly tests this.

11.5 The 77% Could Be Wrong

Project Ariadne’s methodology uses do-calculus interventions, which assume a specific causal model. If the causal model is mis-specified, the 77% figure could overstate decorative token prevalence. Experiment 2C (causal load-bearing analysis within ChatDB traces) provides an independent replication. If our measurements diverge significantly from Ariadne’s, the discrepancy becomes an important finding in itself.

12. Formal Foundations: Radial Restraint and the Obligation Lifecycle

The formal semantics of the obligation engine derive from Grosz and Swift’s radial restraint framework. Radial restraint assigns a third truth value—*undefined*—to atoms exceeding a specified complexity bound. The obligation engine maps this directly: “open obligation” is operationally equivalent to the undefined truth value.

The critical property: radial restraint prevents error propagation through negative dependencies. A proof’s completeness claim depends negatively on every unaddressed case. Standard systems treat unexplored branches as false. The obligation engine treats them as undefined, preventing false claims of completeness. This is the mechanism that catches IMO P6: all systems defaulted unexplored permutation cases to false, producing 4048 instead of recognizing structural incompleteness.

The obligation lifecycle—open → assigned → closed or demoted—is governed by a Generalized Courteous Logic Program (GCLP) with prioritized deontic rules. The prohibition against termination while obligations remain open (R_2) has higher priority than any solver signal. The five-level reassignment cascade (reassign → decompose → escalate → research → council consult) exhausts alternatives before demotion. The approximation chain from radial restraint guarantees monotonic convergence: each pass strictly improves soundness without invalidating previous results.

These guarantees are properties of the tracking architecture, not of any particular foundation model. They hold for any solver—Claude, GPT, Gemini, DeepSeek, open-source—because they depend on the DAG and the obligation invariant, not on the generative process.

13. Research Timeline and Deliverables

Phase	Duration	Activities	Deliverables	Risk De-Risked
Harness + Dataset	Weeks 1–3	Unified runner for IMO/miniF2F/PutnamBench/FormalMATH; obligation-mass metrics; DAG instrumentation	Evaluation harness; benchmark basket; logging infrastructure	Prevents one-off results; enables ablations
Phase I: Empirical	Months 1–8	Experiments 1A–4C; budget scaling sweeps; cross-model error correlation; causal load-	Dataset of discovery rates, switching costs, load-bearing fractions;	Tests whether signatures are stable curves vs. anecdotal

		bearing analysis; Scalpel pattern impact measurement	four empirical papers	
Phase II: Theoretical	Months 6–16	Formal impossibility arguments; recursive obligation opacity proof; boundary characterization; connection to 77% training signal	Four theoretical papers; boundary conditions; Track B theoretical framework	Converts observations into provable claims
Phase III: Architectural	Months 12–24	Obligation engine refinement; three-substrate stack optimization; Scalpel pattern integration; open-source toolkit	Apache 2.0 release; ChatDB implementation; DoD domain demonstrations	Validates on defense-relevant domains
Integration	Months 18–24	Cross-cutting synthesis; Consent Horizon mapping; muda framework formalization; unified paper	Capstone: The Four Irreducibilities; conference submission	Establishes field-level contribution

Phases overlap deliberately. Each produces independent publishable results. Estimated effort: 1–2 engineers + 1 research lead. Compute scales with API budget; the obligation engine’s per-problem cost of \$40–60 is three orders of magnitude below frontier training runs.

14. Falsifiability Matrix

Each irreducibility is a claim, not an axiom. We commit to reporting negative results.

Irreducibility	Supported If...	Falsified If...	Current Weight
Obligation Opacity (recursive)	Root and node-level discovery ratio >0.2 for frontier models; human experts show similar patterns	A computable function enumerates all obligations (root and recursive) without exploration, ratio <0.05 across all classes	High: theorems + empirical + architecture
Solver Opacity	Prediction accuracy from output history plateaus below 0.85; load-bearing fraction ~23%; cross-model error correlation low	Interpretability method achieves >0.9 causal prediction on novel steps; OR load-bearing fraction >0.7	High: NP-hardness + 77% decoupling
V-E Tradeoff	Pareto frontier strictly concave; waste-eliminated architecture operates closer to frontier than standard; even oracle cannot achieve both maxima	Verification with zero marginal cost; OR waste-eliminated and standard architectures converge to same frontier position	High: tight Pareto theorem + production evidence
Strategy	Switching costs strictly	Scheduler achieves	High: $T^1, \rightarrow T^2$ theorem +

Interference	positive; independent parallel is only elimination; context contamination measurable in conversation	Pareto optimality across coverage, depth, and efficiency for arbitrary trees	universal context rot
--------------	--	--	-----------------------

15. Connection to the Consent Horizon Framework

The four proof-search irreducibilities are instances of the four governance irreducibilities identified in *The Consent Horizon* (Vario, 2026). The mapping is structural, not analogical: both are coordination problems among agents under bounded resources, and the same impossibilities bind them.

Governance Irreducibility	Proof-Search Instance	Shared Structure	Impossibility Source
Lossy Intent	Obligation Opacity	Representation compresses; the map is never the territory	Information-theoretic: compression destroys
Concurrent Interests	Strategy Interference	Legitimate alternatives compete for finite attention	Arrow-type: no fair aggregation without dictator
Temporal Validation Cost	V-E Tradeoff	Trust-building consumes the resource it protects	Bandit theory: exploration costs exploitation
Incomplete Self-Knowledge	Solver Opacity	No agent fully models its own process	Complexity: self-interpretation is NP-hard

The deeper claim: these are not four analogies but four instances of the same four constraints expressed in different substrates. If confirmed empirically and theoretically, they support the Consent Horizon’s central thesis: the project of building trustworthy AI and the project of building legitimate governance are architecturally identical—the same structural constants govern both.

16. Conclusion

This document establishes the Four Irreducibilities of Neural Proof Search as falsifiable claims grounded in proven impossibility theorems, supported by convergent evidence, and connected to a quantified waste-elimination framework spanning training, inference, and infrastructure.

The practical implication is direct: AI reasoning systems become more trustworthy not by eliminating their limitations, but by knowing them. A system that knows its obligation extraction is incomplete and refuses to claim otherwise; that separates its solver from its verifier because it cannot trust its own process; that makes its verification-exploration allocation visible; that documents strategy failures for future reference; and that eliminates 77% waste through architectural constraint rather than scale—such a system is more legitimate than one that claims unbounded capability.

The database holds the state. The agents are hands. The orchestration layer decides what each hand sees. Everything persists; only what matters activates. This is the separation of storage from attention, and it is the architectural response to four proven constraints on what neural systems can do.

Perfect proof is unreachable. Perfect consent is unreachable. Systems that know this, and make their approach velocity visible rather than claiming arrival, are both reachable and urgently needed.

References

Impossibility Theorems and Foundational Results

- [1] Statman, R. (1979). Lower bounds on Herbrand’s theorem. *Proceedings of the AMS*, 75(1), 104–107.
- [2] Boolos, G. (1984). Don’t eliminate cut. *Journal of Philosophical Logic*, 13(4), 373–378.
- [3] Buss, S. (2012). Sharpened lower bounds for cut elimination. *Journal of Symbolic Logic*, 77(2), 656–668.
- [4] Rice, H.G. (1953). Classes of recursively enumerable sets and their decision problems. *Trans. AMS*, 74, 358–366.
- [5] Baldan, P., Ranzato, F., & Zhang, L. (2021). A Rice’s theorem for abstract semantics. *Proc. ICALP 2021, LIPIcs* 198.
- [6] “Don’t Eliminate Cut: Exponential Separations in LLM-Based Theorem Proving.” *arXiv:2602.10512*, February 2025.
- [7] Wolpert, D.H. & Macready, W.G. (1997). No free lunch theorems for optimization. *IEEE Trans. EC*, 1(1).
- [8] Barceló, P., Monet, M., Pérez, J., & Subercaseaux, B. (2020). Model interpretability through the lens of computational complexity. *NeurIPS 2020*.
- [9] Adolphi, F. et al. (2024). The computational complexity of circuit discovery for inner interpretability. *arXiv:2410.08025*.
- [10] “The Limits of AI Explainability: An Algorithmic Information Theory Approach.” *arXiv:2504.20676*, 2025.
- [11] Rudin, C. (2019). Stop explaining black box machine learning models. *Nature Machine Intelligence*, 1, 206–215.
- [12] Chen, Y. et al. (2026). Bandit Allocational Instability. *arXiv:2602.07472*, February 2026.
- [13] Lai, T.L. & Robbins, H. (1985). Asymptotically efficient adaptive allocation rules. *Adv. Applied Mathematics*, 6(1), 4–22.
- [14] Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47, 235–256.
- [15] Dekel, O., Ding, J., Koren, T., & Peres, Y. (2014). Bandits with switching costs: T^2 regret. *STOC 2014*.
- [16] Koren, T., Livni, R., & Mansour, Y. (2017). Multi-armed bandits with metric movement costs. *NeurIPS 2017*.
- [17] Amir, I. et al. (2022). Better best of both worlds bounds for bandits with switching costs. *NeurIPS 2022*.
- [18] Arrow, K.J. (1951). *Social Choice and Individual Values*. Wiley.
- [19] Orevkov, V.P. (1979). Lower bounds for increasing complexity of derivations after cut elimination. *Zapiski Nauchnykh Seminarov LOMI* 88.

Empirical Evidence and AI Reasoning Systems

- [20] Project Ariadne (2025). Causal decoupling in LLM reasoning traces. *arXiv:2601.02314*.
- [21] Bochkov, I. et al. (2025). Emergent semantics beyond token embeddings. *TMLR 2025 / arXiv:2507.04886*.
- [22] Anthropic (2025). Circuit tracing: Revealing computational graphs in language models. *transformer-circuits.pub*.
- [23] Liu, N.F. et al. (2024). Lost in the middle: How language models use long contexts. *TACL 2024 / arXiv:2307.03172*.
- [24] Chroma Research (2025). Context rot: How increasing input tokens impacts LLM performance.
- [25] Snell, C. et al. (2024). Scaling LLM test-time compute optimally. *arXiv:2408.03314*.
- [26] Slack, D. et al. (2020). Fooling LIME and SHAP. *AIES 2020 / arXiv:1911.02508*.
- [27] Huang, J. et al. (2024). Large language models cannot self-correct reasoning yet without external feedback. *ICLR 2024*.
- [28] Lightman, H. et al. (2023). Let’s verify step by step. *OpenAI / arXiv:2305.20050*.

Proof Systems and Benchmarks

- [29] Google DeepMind (2025). Gemini with Deep Think achieves gold medal at IMO 2025.
- [30] ByteDance Seed (2025). Seed-Prover 1.5. arXiv:2507.23726.
- [31] DeepSeek (2024). DeepSeek-Prover-V1.5. arXiv:2408.08152.
- [32] Erbsen, A. et al. (2024). Towards a scalable proof engine. JAR.
- [33] Wells, J.B. (1999). Typability and type checking in System F are equivalent and undecidable. *Annals of Pure and Applied Logic*, 98(1–3).
- [34] Xu, L. et al. (2012). SATzilla: Portfolio-based algorithm selection for SAT. *JAIR* 32.
- [35] Knuth, D.E. & Moore, R.W. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4), 293–326.
- [36] Yokoo, M. (1994). Weak-commitment search for solving constraint satisfaction problems. *AAAI-94*.
- [37] Eck, B.T. & Pinedo, M. (1993). On the minimization of the makespan subject to flowtime optimality. *Operations Research*, 41(4).

Logic Programming and Formal Semantics

- [38] Grosz, B.N. & Swift, T. (2013). Radial restraint: A semantically clean approach to bounded rationality for logic programs. *AAAI* 2013.
- [39] Grosz, B.N. & Swift, T. (2013). Technical report with proofs. *Coherent Knowledge Systems*.
- [40] Van Gelder, A., Ross, K., & Schlipf, J. (1991). The well-founded semantics for general logic programs. *JACM*, 38(3).
- [41] Grosz, B.N., Labrou, Y., & Chan, H.Y. (1999). A declarative approach to business rules in contracts. *EC-99*.
- [42] Wan, H., Grosz, B.N., Kifer, M., Fodor, P., & Liang, S. (2009). Logic programming with defaults and argumentation theories. *ICLP* 2009.
- [43] Kohlenbach, U. (2008). *Applied Proof Theory*. Springer.

Governance, Cross-Cutting Theory, and Infrastructure

- [44] Vario (2026). The Consent Horizon: Perfect consent as the asymptotic limit of governance architecture. Working paper, Vario Automation.
- [45] Russell, S.J. (1997). Rationality and intelligence. *Artificial Intelligence*, 94(1–2), 57–77.
- [46] Genewein, T. et al. (2015). Bounded rationality, abstraction, and hierarchical decision-making. *Frontiers in Robotics and AI*, 2, 27.
- [47] Brcic, M. & Yampolskiy, R. (2023). Impossibility results in AI: A survey. *ACM Computing Surveys*, 56(1).
- [48] Wolfram, S. (2002). *A New Kind of Science*. Chapter 12: Computational Irreducibility.
- [49] Conitzer, V. & Sandholm, T. (2004). Computational social choice. *Handbook of Computational Social Choice*.
- [50] Mnehmos (2025). The Scalpel, Not the Hammer: Five patterns from production MCP servers. validatedintent.com/blog.

• • •

Correspondence: Vario, Vario Automation, Graham County, Arizona. validatedintent.com